Machine Learning Operations (MLOps)

AIMLCZG523 Assignment 01

CardiaScreen: an end-to-end screening model, tracked experiment, and monitored API

Student name: Ankit Kumar Srivastava BITS ID: 2025AE05293

Submitted by Ankit Kumar Srivastava (2025AE05293). The repository, figures, and model artifact were produced for this assignment from the official UCI Heart Disease files.

1. Problem and result

The task is a binary classifier for angiographic heart disease. A row is positive when the UCI *num* field is greater than zero, which means at least 50 percent diameter narrowing. The delivered system trains on the four public cohorts, freezes one model after a training-only comparison, packages the preprocessing with the estimator, and serves it from a container that logs every request and exposes Prometheus metrics.

The selected model is L2 logistic regression with C = 0.1. Five-fold ROC-AUC on the training split is 0.895 (standard deviation 0.013). On the untouched 20 percent holdout the ROC-AUC is 0.909 and average precision is 0.915. The operating threshold 0.18 was chosen to maximize F2 on training out-of-fold scores, so holdout recall is 0.980 and precision is 0.694. At the default threshold of 0.50, holdout accuracy is 0.832 with precision 0.851 and recall 0.843. This score is a coursework screening signal. It is not a diagnosis and it is not a medical device.

2. How to reproduce the run

From a clean checkout, create a Python 3.11 environment and install the pinned files. `requirements.txt` trains and tracks. `requirements-api.txt` is the smaller set baked into the serving image. `environment.yml` is the same stack for Conda. PowerShell:

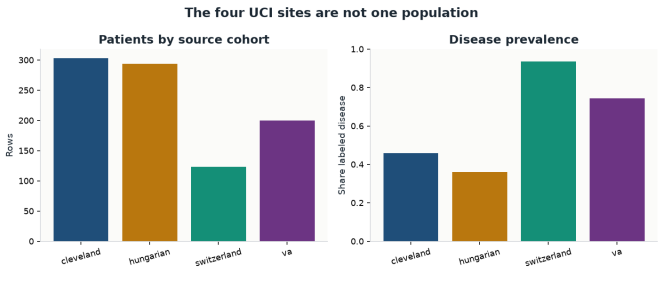
```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt -r requirements-dev.txt
$env:PYTHONPATH = "src"
python -m cardiascreen.data
python -m cardiascreen.eda --skip-download
python -m cardiascreen.train --skip-download
pytest -q
docker compose up --build
```

Training writes `models/heart_risk_model.joblib`, `reports/metrics.json`, the figures in `reports/figures`, and an MLflow sqlite store at `mlruns/mlflow.db`. Open the tracking UI with `mlflow ui --backend-store-uri sqlite:///mlruns/mlflow.db`. MLflow 3 no longer treats a plain folder as a supported tracking backend, so the run uses sqlite and still keeps artifacts on disk. The random seed is 42.

3. Data acquisition and cleaning

The source is the UCI Heart Disease collection (Janosi, Steinbrunn, Pfisterer, and Detrano, 1988; DOI 10.24432/C52P4X). `python -m cardiascreen.data` downloads the four processed files from the UCI repository: Cleveland (303 rows), Hungary (294), Switzerland (123), and the VA Long Beach file (200). The combined table has 920 labeled rows and a positive rate of 0.553. Question marks are missing values. Cholesterol, resting blood pressure, and maximum heart rate stored as zero are also treated as missing. Switzerland's cholesterol column is entirely zero in the source file, so after cleaning it is entirely missing. That choice matters: a model that treated those zeros as real measurements would learn a false biological rule.

The label is binary. Severity codes 1 through 4 are collapsed to disease present, and code 0 stays disease absent. Site identity is kept for evaluation and is not given to the model. A random split that can see the hospital learns the hospital's base rate. Cleveland and Hungary sit near 45 and 36 percent positive. Switzerland is above 90 percent and the VA file is about 75 percent. Those are different populations.



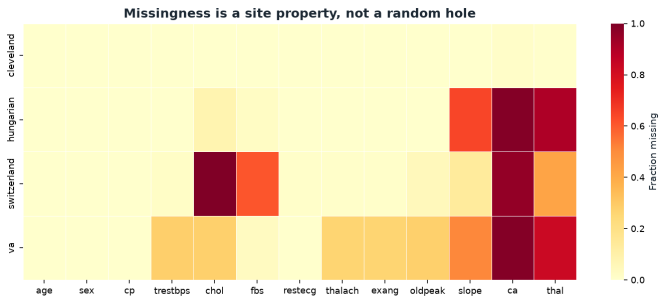


Figure 1. Cohort size and disease prevalence. The sites do not share a base rate. Figure 2. Missingness by site. Vessel count (ca) is almost absent outside Cleveland, and cholesterol is missing for every Switzerland row.

4. Exploratory findings

Exploratory plots describe the full assembled table. They were not used to pick the winning hyperparameters. Those were chosen on the training split only. The correlation heatmap is restricted to Cleveland, because correlating the pooled table would mostly measure which site forgot which test. In Cleveland, disease moves with lower maximum heart rate, higher ST depression, more colored vessels, and exercise-induced angina. Age and resting blood pressure move together, and cholesterol is only weakly tied to the label. That is why the later model is allowed to use cholesterol, but the report does not treat it as the main signal.

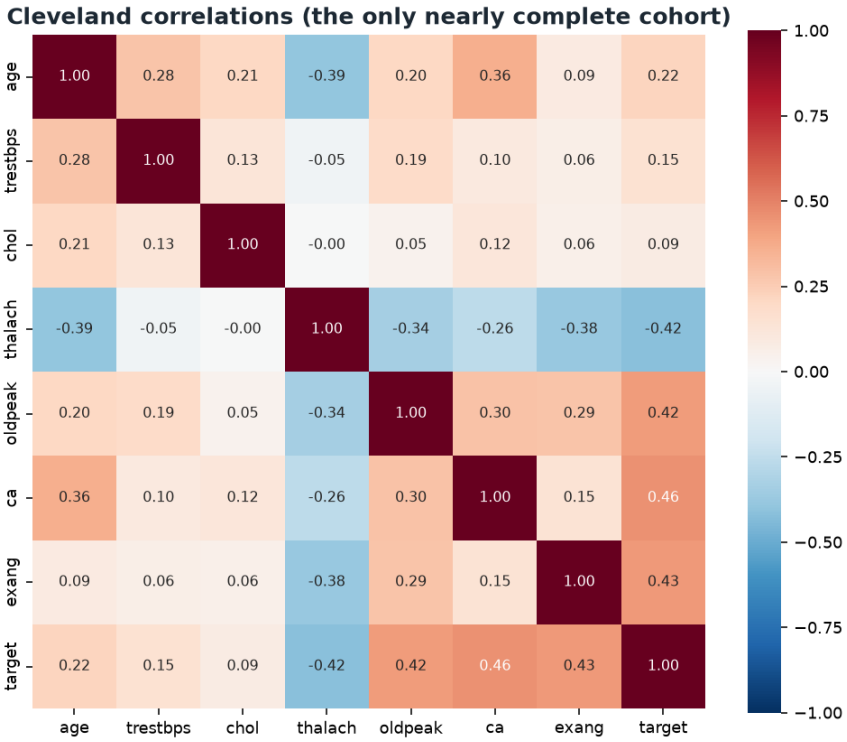


Figure 3. Cleveland-only correlations. The other sites are left out so that missingness patterns do not masquerade as clinical correlation.

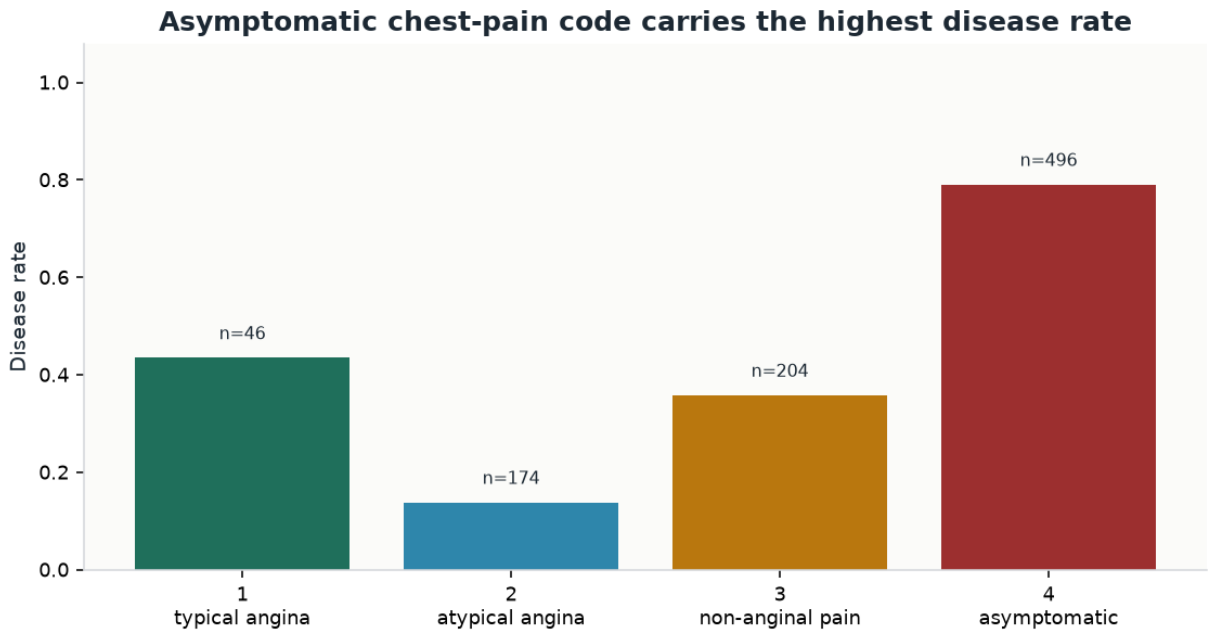


Figure 4. UCI chest-pain codes. Code 4 (asymptomatic) is the largest group and the highest disease rate. The API rejects code 0 so a Kaggle-style payload cannot be scored.

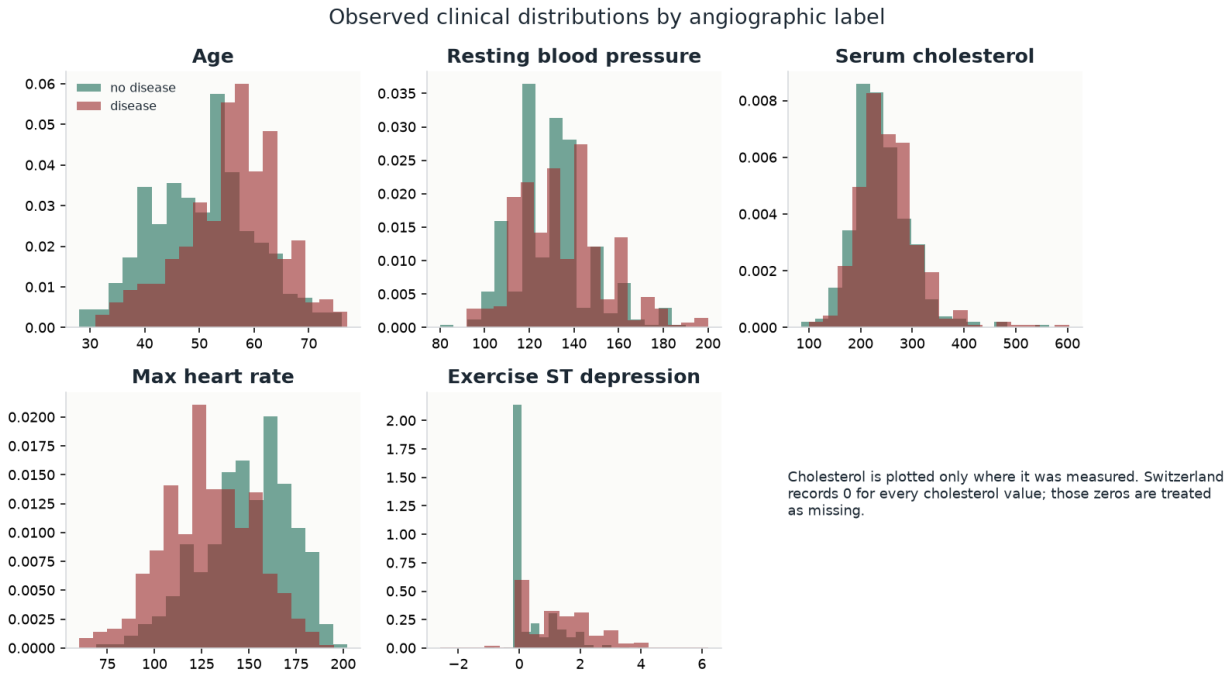


Figure 5. Disease rows are older, reach a lower peak heart rate, and show more ST depression. Cholesterol overlaps heavily once the zero sentinels are removed.

5. Feature engineering

The serving path and the training path share one sklearn transformer, `ClinicalFeatureBuilder`. It reads the thirteen raw UCI fields, repeats the zero-sentinel rule, and adds four clinical fields: heart rate as a fraction of the predicted maximum (220 minus age), cholesterol per year of age, blood pressure per year of age, and the product of ST depression and exercise angina. Missingness indicators are added for cholesterol, fasting blood sugar, vessel count, and the thallium code, because 'this test was not done' is known at request time.

Numeric fields are median-imputed and standardized. Binary fields use the most frequent value. Chest pain, resting ECG, ST slope, and thallium code are one-hot encoded with unknown codes ignored. Thallium is a nominal code (3 normal, 6 fixed

defect, 7 reversible defect), so it is not scaled as if it were a measurement. Vessel count stays numeric because it is a count, and it keeps its own missingness indicator. The transformer drops cohort, the raw severity code, and the label if a caller passes them. A unit test locks that rule. Fluoroscopy and thallium are optional on `POST /predict`, since a screening form often will not have those invasive results.

6. Model selection and the operating point

The table is split 80/20, stratified on the label, with seed 42 (736 train rows, 184 holdout rows). Three families are compared with a grid search and five stratified folds, scored by ROC-AUC. The holdout is not used for that choice. Logistic regression uses C in {0.1, 1, 10} with balanced class weights. Random forest varies depth and leaf size around 250 trees. HistGradientBoosting varies depth and learning rate. Logistic regression wins. The forest and the boosting model are close behind and less stable across folds, which is a reason to prefer the linear model on a table of this size.

Model	CV ROC-AUC	Fold SD	Selected parameters
logistic regression	0.895	0.013	C=0.1
random forest	0.881	0.024	max_depth=6, min_samples_leaf=2
hist gradient boosting	0.873	0.022	learning_rate=0.05, max_depth=3

Table 1. Training-split comparison. The winner is the first row. Parameters were not retuned after looking at the holdout.

After the winner is frozen, out-of-fold probabilities on the training split are used to pick a threshold that maximizes F2 (beta = 2). The chosen threshold is 0.18. F2 is the selection rule because a missed narrowing is more costly, in this screening setup, than an extra review. The same holdout probabilities are also reported at 0.50 so the accuracy cost of that choice is visible. Moving from 0.50 to 0.18 raises recall from 0.843 to 0.980 and lowers precision from 0.851 to 0.694. Accuracy falls from 0.832 to 0.750. The confusion matrix at the operating point has 100 true positives and 2 false negatives on 102 holdout disease rows, with 44 false positives.

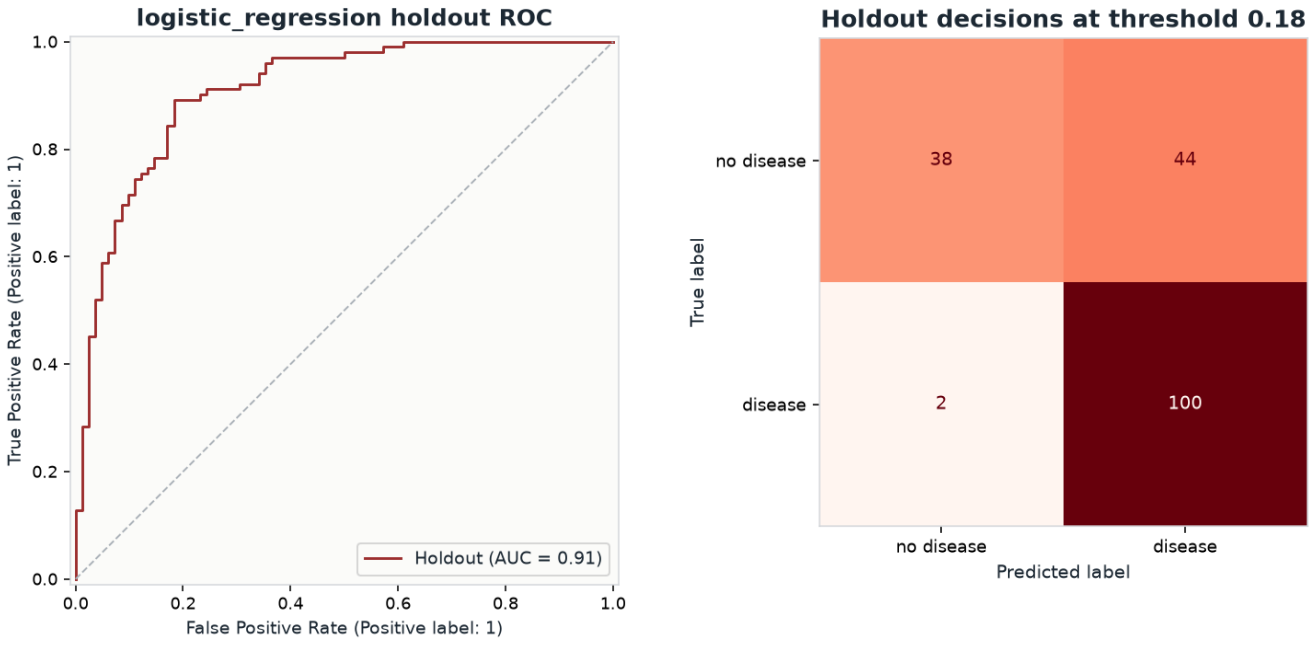


Figure 6. Holdout ROC for the frozen logistic model, and holdout decisions at the F2 threshold.

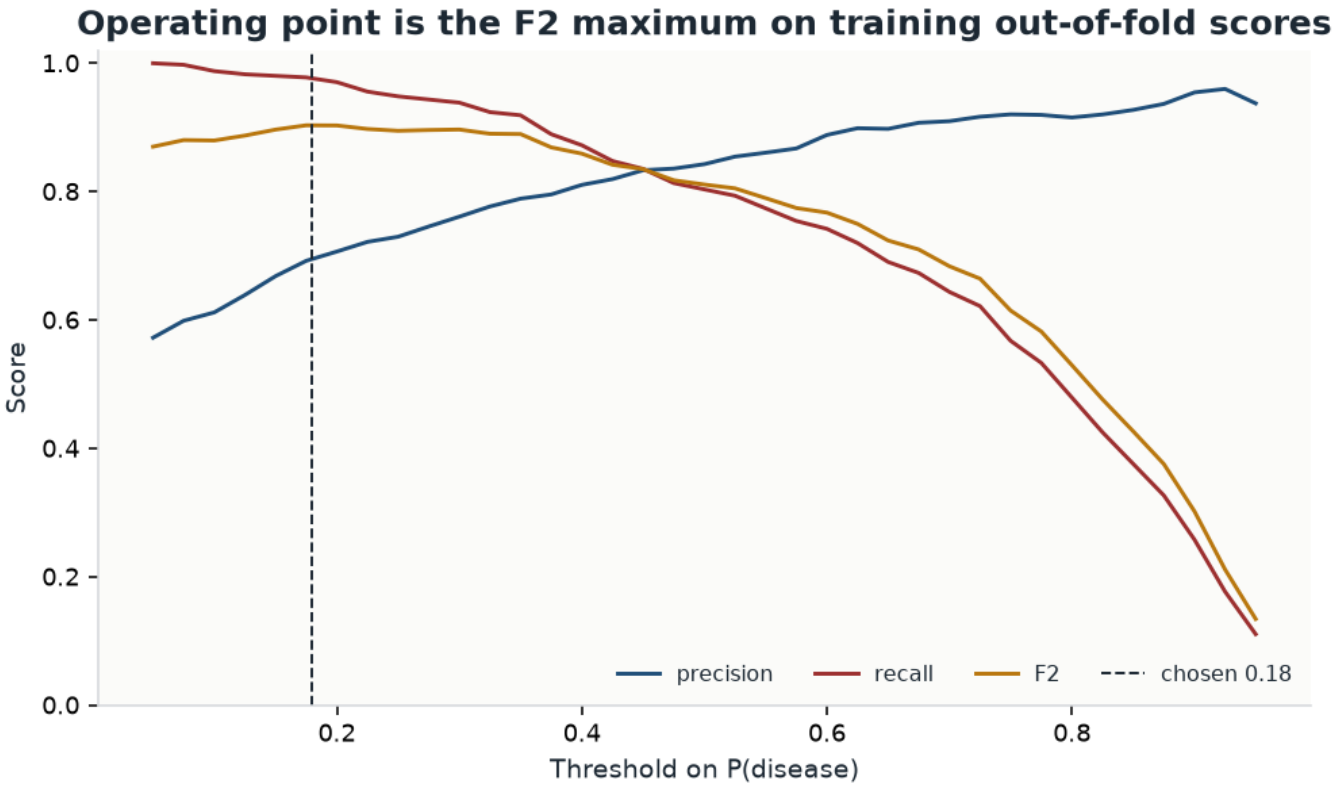


Figure 7. Precision, recall, and F2 against the threshold. The vertical line is the training out-of-fold choice. It is not fit on the holdout.

7. Holdout results, including the sites

Threshold	Accuracy	Precision	Recall	F1	F2	ROC-AUC	Brier
0.18 (F2)	0.750	0.694	0.980	0.813	0.906	0.909	0.121
0.50	0.832	0.851	0.843	0.847	0.845	0.909	0.121

Table 2. One holdout, two operating points. ROC-AUC and Brier do not depend on the threshold.

Holdout site	Rows	Positives	Recall	Precision	ROC-AUC
cleveland	55	26	0.962	0.658	0.930
hungarian	64	22	0.955	0.512	0.911
switzerland	28	27	1.000	0.964	n/a
va	37	27	1.000	0.730	0.767

Table 3. The same frozen model and the same threshold, sliced by source site. ROC-AUC is omitted when either class has fewer than five rows. The Switzerland holdout slice is almost entirely positive, so an AUC there would be noise. High accuracy on that slice mostly repeats the base rate. Hungary, where disease is less common, is where the low threshold spends precision.

A separate transport check fits the same algorithm and the same hyperparameters on Cleveland only, then scores the other three sites in full. This model is not the production artifact and it did not vote in model selection. Ranking still travels: Hungary's ROC-AUC stays near 0.89. Recall at 0.50 does not travel. Hungary recall is about 0.60, Switzerland about 0.60, and the VA file about 0.50. The production model is therefore the pooled model, because it has seen the missingness patterns a new request will actually have. The threshold would still need a local review before use in a clinic whose prevalence differs from this pool.

External site	Rows	ROC-AUC	Recall at 0.50	Precision at 0.50
hungarian	294	0.890	0.604	0.865
switzerland	123	0.740	0.600	0.972
va	200	0.709	0.497	0.860

Table 4. Cleveland-only model scored on the other UCI sites. Transport of ranking is better than transport of a fixed 0.50 cutoff.

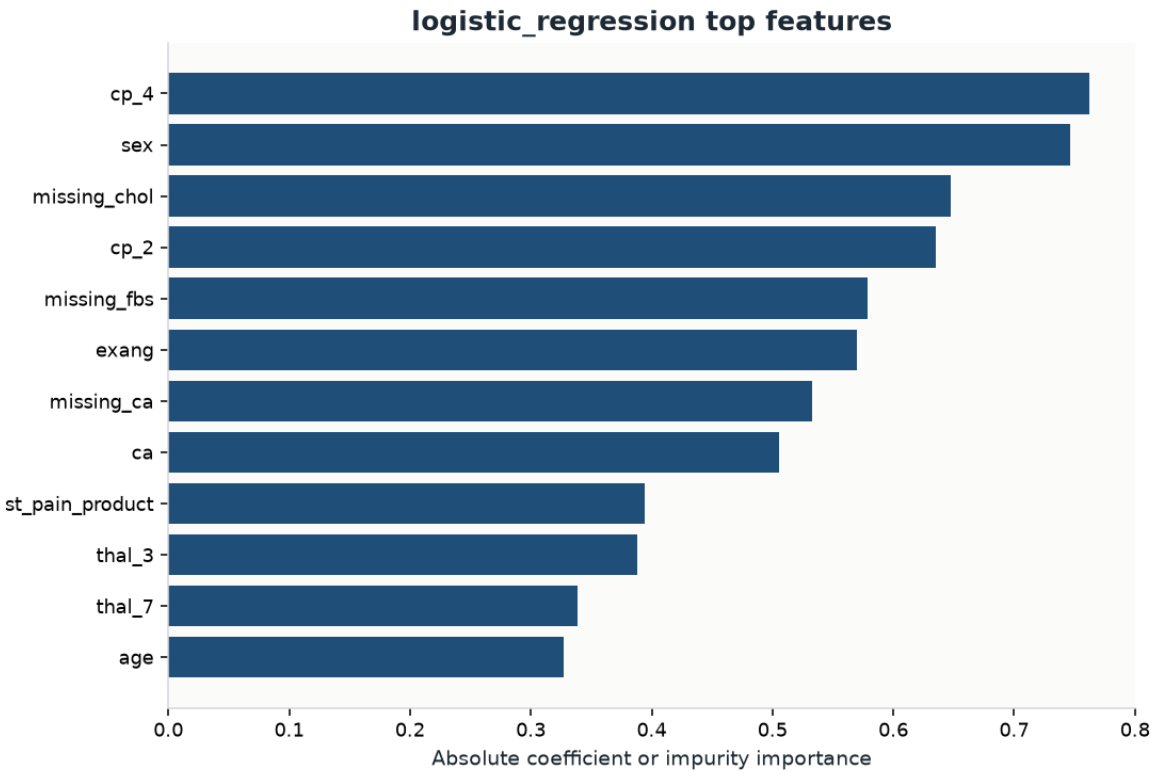


Figure 8. Largest absolute coefficients. Asymptomatic chest pain and sex lead. A positive sign raises the log-odds of disease.

Feature	Signed coefficient
cp_4	+0.762
sex	+0.746
missing_chol	+0.647
cp_2	-0.635
missing_fbs	+0.579
exang	+0.569
missing_ca	+0.533
ca	+0.505

Table 5. Signs for the largest coefficients of the packaged logistic model. missing_chol has a large coefficient because Switzerland, a high-prevalence site, contributes an entirely missing cholesterol column. The indicator is partly a stand-in for that site. A clinic that always measures cholesterol will not activate it. This is the main leakage risk left after dropping the cohort column, and it is why the API metrics include the predicted-probability histogram: a shift in missingness shows up as a shift in scores.

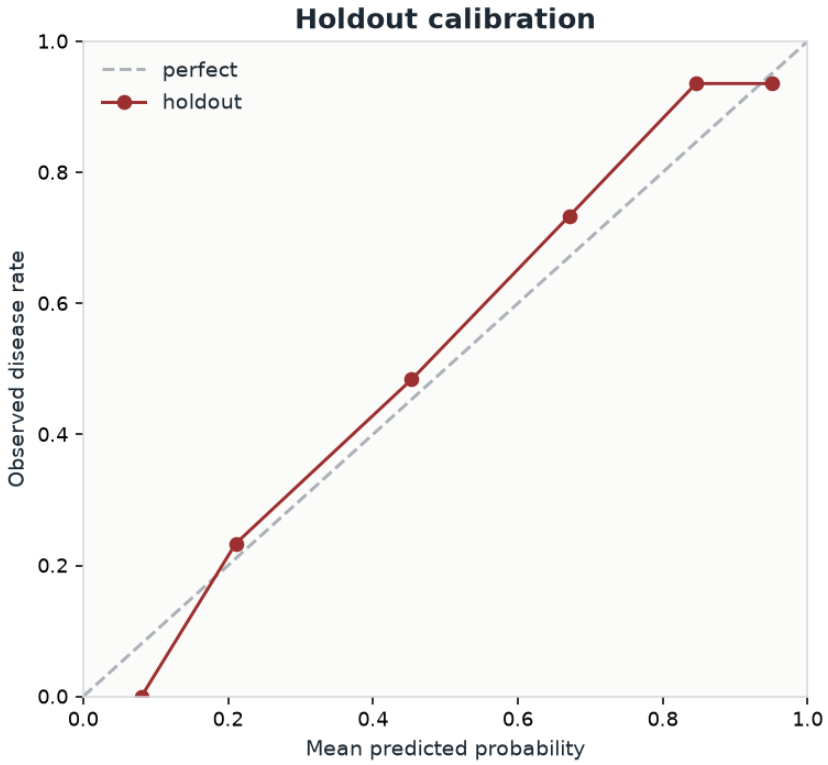


Figure 9. Holdout calibration. Ranking can be good while a probability is still a poor frequency, which is why the API returns both a probability and a confidence.

8. Experiment tracking and packaging

Each candidate is a nested MLflow run under the experiment `cardiascreen-heart-disease`. The parent run records the split sizes, the seed, the selection metric, the threshold rule, the winner, the holdout metrics, the figure directory, the metrics file, and the sklearn model. The custom transformer is declared in the skops trusted-type list so the pyfunc save can record it. The file that the API actually loads is a joblib dictionary: the fitted pipeline, the threshold, the version, the training time, the metric snapshot, and a disclaimer. Loading that file requires the package path `cardiascreen.features`, which the image provides through `PYTHONPATH=/app/src`. Code and model ship in one image, so a release is one digest rather than a model file that can drift away from its transformer.

Inference without HTTP is `python -m cardiascreen.infer --input deploy/sample_request.json`. The three sample files cover a high-risk Cleveland-style row, a lower-risk row, and a screening row that omits cholesterol, fasting sugar, vessel count, and thallium.

9. API, tests, and the CI pipeline

FastAPI exposes `GET /health`, `GET /metadata`, `GET /metrics`, and `POST /predict`. The body is validated with Pydantic. Extra fields are rejected, including a client-supplied label. Chest-pain values outside 1-4 are rejected. The response returns the class, the probability of disease, the confidence in the emitted class, the threshold, a risk band (low, moderate, high, very high, cut at 0.25, 0.50, and 0.75), a request id, and the disclaimer. Each decision is a JSON log line on stdout and, when the directory is writable, in `logs/api.jsonl`. The log records age, sex, chest-pain code, the decision, and latency. A production deployment would redact even those fields.

Pytest covers label construction, zero sentinels, the four-site load, the feature contract, the threshold helper, the small-class AUC guard, and the HTTP contract including the Kaggle chest-pain rejection. GitHub Actions in `.github/workflows/ci.yml` runs ruff, then pytest, then a full retrain, and uploads the model, metrics, and figures. A second job builds the image and calls `/health` and `/predict`. The workflow does not set continue-on-error. A lint failure, a test failure, or a training exception fails the job and the logs stay in the Actions run. The container job uses the model committed in the repository. The retrain job proves that a clean runner can rebuild it.

10. Container, deployment, and monitoring

The Dockerfile is a Python 3.11 slim image running as a non-root user. It installs only the API requirements, copies the package and the joblib artifact, and starts uvicorn on port 8000 with a factory so the model is loaded at process start. A failed load stops the process, which is what a Kubernetes liveness probe should see. `docker compose up --build` starts the API, Prometheus, and Grafana. Prometheus scrapes `api:8000/metrics` every five seconds. Grafana is provisioned with a Prometheus datasource and a dashboard for request rate, latency, decision counts, and the probability histogram. Anonymous viewing is enabled. The admin password in the compose file is `admin` and is only for this local lab.

Kubernetes manifests under `deploy/k8s` define a namespace, a two-replica Deployment with requests and limits, readiness and liveness probes on `/health`, a LoadBalancer Service on port 80, and an nginx Ingress for `cardiascreen.local`. Pods carry Prometheus scrape annotations. Apply them with `kubectl apply -k deploy/k8s` after `docker build -t cardiascreen-api:local .`. Docker Desktop's kubectl context is present on the development machine, and the API server on port 6443 was not running when this report was written, so the verified runtime is the compose stack rather than a live cluster. Enabling Kubernetes in Docker Desktop, or loading the image into minikube, uses these same manifests. If the LoadBalancer address stays pending, `kubectl port-forward svc/cardiascreen-api 8080:80 -n cardiascreen` is the check.

`scripts/export_space.py` copies the Dockerfile, the API requirements, the package, the model, and a Space README into `dist/huggingface-space`. That folder is a Hugging Face Docker Space on the free CPU tier. Upload it with the Hugging Face CLI after `huggingface-cli login`. The Space sleeps when idle; the first request after sleep is slow. The public URL, once created, is the deployed API link for the submission.

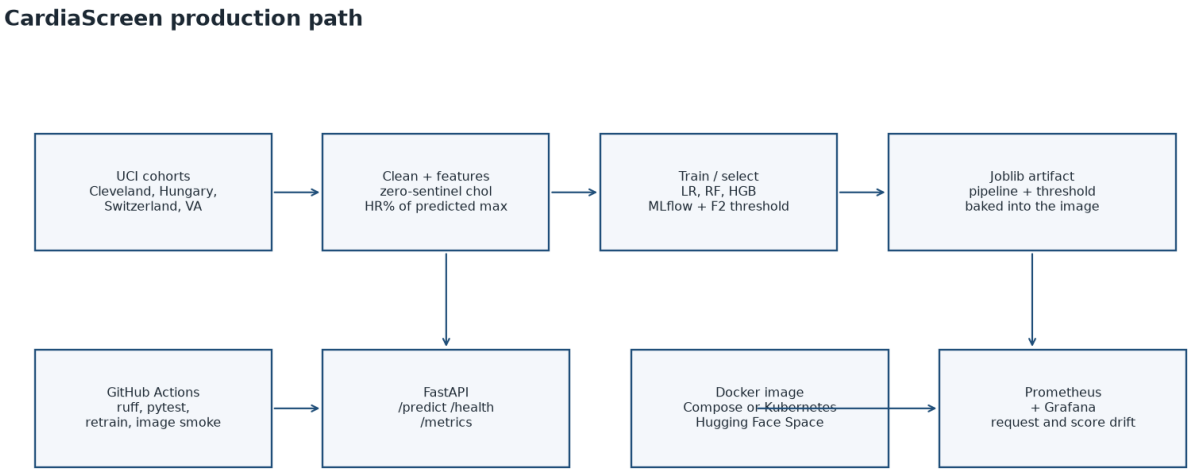


Figure 10. Production path. GitHub Actions gates the image. The image pins code and model together. Prometheus watches the running API.

11. Evidence from the running stack

The figures below are from the local compose stack on 9 October 2026, after the image was built and the API had served prediction requests. `screenshots/` also holds the health response and the raw metrics page. GitHub Actions checks the data pin, then hands a model over only when the holdout gate passes (`screenshots/ci_actions.png`). The workflow is `.github/workflows/ci.yml`.

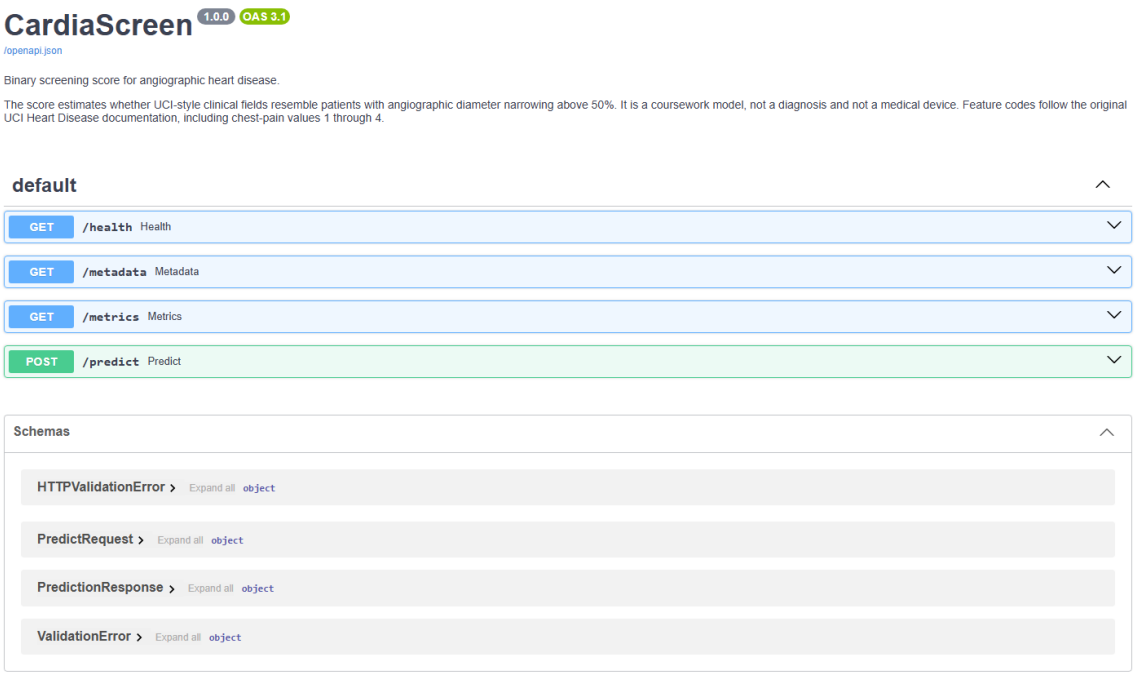


Figure 11. Interactive API documentation at http://127.0.0.1:8000/docs, including POST /predict.

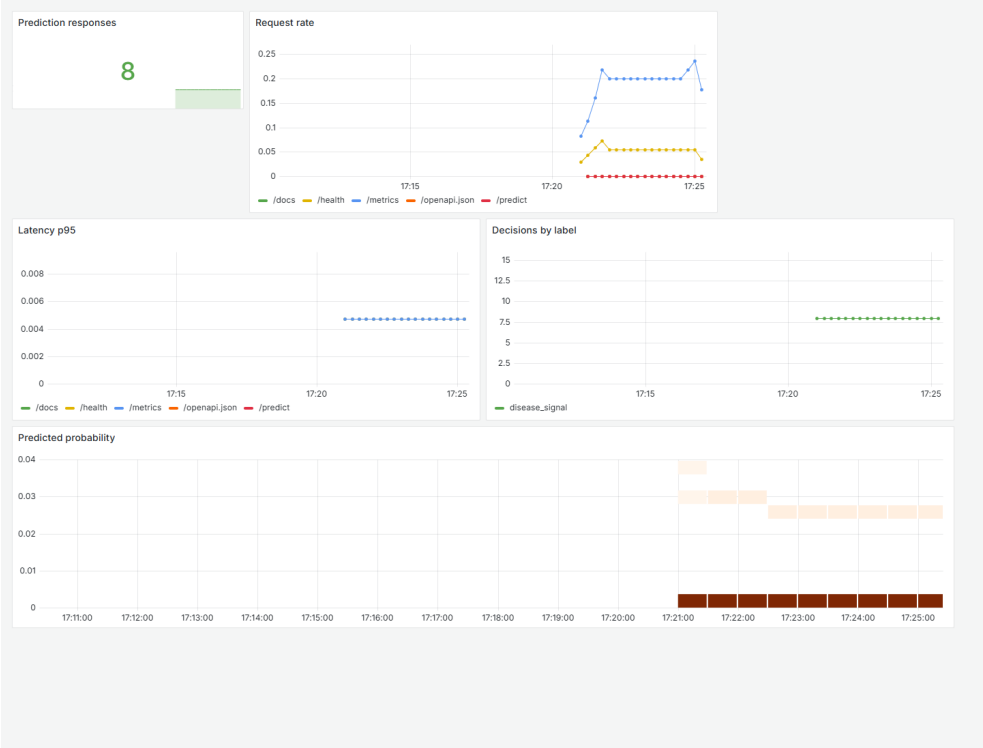


Figure 12. Grafana board provisioned for the API. The prediction counter reads 8 after the smoke requests used to check the stack.

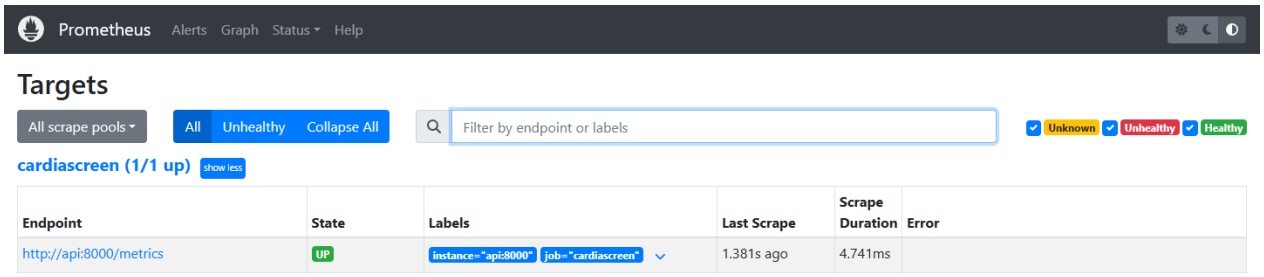


Figure 13. Prometheus target http://api:8000/metrics is UP for job cardiascreen.

12. What this submission does not claim

The holdout is a random split of a 920-row public table, not a prospective clinic. Switzerland and the VA file are small, and their labels are heavily imbalanced, so site-level accuracy is easy to over-read. The missing-cholesterol coefficient is a site proxy and should be watched if the measurement process changes. Probabilities are good enough to rank (holdout Brier 0.121) but the calibration plot in `reports/figures/model_calibration.png` should be read before anyone treats a probability as a frequency. No fairness audit by sex was used as a release gate, even though sex is one of the largest coefficients. That would be the next experiment, not a claim already made.

A short screen recording is part of the deliverable and has to be captured on the machine that runs the pipeline. `docs/DEMO_SCRIPT.md` is a three-minute shot list: the training command, the MLflow UI, a prediction, the Prometheus target, and the Grafana board. Record it after the compose stack is up.

13. Where each graded part lives

Task	Where to look
1. Data and EDA	cardiascreen.data, cardiascreen.eda, reports/figures/eda_*.png, notebooks/
2. Features and models	cardiascreen.features, cardiascreen.train, reports/metrics.json
3. Experiment tracking	mlruns/mlflow.db, nested runs under cardiascreen-heart-disease
4. Packaging	models/heart_risk_model.joblib, requirements.txt, environment.yml
5. Tests and CI	tests/, .github/workflows/ci.yml (ruff, pytest, retrain, image smoke)
6. Container API	Dockerfile, POST /predict, scripts/smoke_predict.py
7. Deployment	docker-compose.yml, deploy/k8s, scripts/export_space.py
8. Monitoring	GET /metrics, Prometheus, Grafana, screenshots/
9. This report	README.md, docs/DEMO_SCRIPT.md, and this PDF

Table 6. Map from the assignment tasks to the files a reviewer can open. The screen recording is the one item that has to be made after this PDF: follow docs/DEMO_SCRIPT.md once the compose stack is running.

Repository layout: `src/cardiascreen` (data, features, training, API), `tests`, `notebooks`, `data/raw` and `data/processed`, `.github/workflows/ci.yml`, `Dockerfile`, `docker-compose.yml`, `deploy/k8s`, and `monitoring`. Dataset citation: Janosi, A., Steinbrunn, W., Pfisterer, M., and Detrano, R. (1988). Heart Disease. UCI Machine Learning Repository. https://doi.org/10.24432/C52P4X.